

СО АН СССР



ВЫЧИСЛИТЕЛЬНЫЙ ЦЕНТР
НОВОСИБИРСК

256 - 114

Л.С.Бараз, Е.В.Боровиков, Н.Г.Глаголева,
П.А.Земцов, Е.В.Налимов, В.А.Цикоза

ЯЗЫК ПРОГРАММИРОВАНИЯ РАПИРА

767

препринт

Академия наук СССР Сибирское отделение
В ы ч и с л и т е л ь н ы й ц е н т р

Л.С.Бараз, Е.В.Боровиков, Н.Г.Глаголева,
П.А.Земцов, Е.В.Налимов, В.А.Цикоза
ЯЗЫК ПРОГРАММИРОВАНИЯ РАПИРА

П р е п р и н т

767

Новосибирск 1987

1. Обзор на примерах

Рапира — язык программирования [1, с. 199-206], разработанный Г.А. Звенигородским как средство перехода от учебного языка программирования Ровик [1] к стандартным языкам программирования. Один из интерпретаторов для этой версии языка был реализован на ПЗВМ Агат в системе "Школьника" [2]. Опыт использования системы "Школьника" позволил начать разработку новой версии языка. Система программирования для новой версии Рапиры реализуется на ПЗВМ Yamaha и Корвет.

В настоящей статье приведен обзор новой версии языка. Язык показан на примерах протоколов работы небольших программ, иллюстрирующих в основном работу с объектами (вывод программы отмечен символом '♦'). Предписания языка Рапира типичны для языков процедурного типа, поэтому в примерах они используются почти без комментариев. (Комментарии в Рапире записываются от символа '/' до конца строки. В настоящей статье комментарии, относящиеся к программе, а не к конструкциям языка, дополнительно выделены символом '■').

1.1. Работа в системе Рапира

Программ на Рапире — при работе в системе — фактически нет. Есть описания процедур и функций и есть отдельные предписания, исполняемые в диалоге. Исполнение предписаний идет в рабочей области памяти. Рабочая область создается при входе в

систему и может быть запомнена при выходе из нее. В рабочую область включаются, в частности, все глобальные переменные со значениями-объектами.

1.2. Объекты и операции над ними

Объекты: логические (да, нет), целые и вещественные числа, тексты, кортежи, пустой объект (пусто). Объектами также считаются процедуры и функции, модули и устройства. Для всех объектов определены операции = и /=.

Логические объекты: да, нет. Операции над ними: и, или, не.

Числа:	125	\ целое
	530.84	\ вещественное
	1.9e-8	\ вещественное

Операции над числами: + и - (унарные и бинарные), *, /, ** (возведение в степень), // и /% (деление нацело и остаток от деления нацело - для целых чисел), >, <, >=, <=.

Пример 1.

```
функ ПРОСТОЕ(N) \ N - простое число или нет?
если N<2 то
    возврат нет \ возврат значения из функции
все
для M от 2 до корень(N)+0.5 цикл
    если N /% M = 0 то
        возврат нет
    все
кц
возврат да
конец
вывод: ПРОСТОЕ(2003) \ вывод на экран
♦ да
```

Тексты:	"В одном сказочном городе жили коротышки"
	"" \ пустой текст
	" 2 * 2 = 11 "

Операции над текстами: # (длина), + (склеивание), * (умножение на целое неотрицательное число), [] (выборка символа), [:] (вырезка подтекста).

Пример 2.

проц РАМКА (N)	\	печать рамки N x N
вывод: "e" x N	\	еееее
повтор (N-2) цикл		
вывод: "e" + " " x (N-2) + "e"	\	e e
кц		
вывод: "e" x N	\	еееее
конец		
вызов РАМКА (5)		
♦ еееее		
♦ e e		
♦ e e		
♦ e e		
♦ еееее		

Пример 3.

проц ШИВОРОТ_НАВЫВОРОТ ()	\	печать текста наоборот
вывод: "Введите текст"	\	приглашение ко вводу
ввод текста: ТЕКСТ	\	запрос текста в диалоге
НАОБОРОТ:= ""	\	пустой текст
для К до #ТЕКСТ цикл	\	от 1
НАОБОРОТ:= ТЕКСТ[К] + НАОБОРОТ	\	#"огурец" = 6
кц	\	"огурец"[3] = "y"
вывод: НАОБОРОТ	\	вывод результата на экран
конец		
вызов ШИВОРОТ_НАВЫВОРОТ()		
♦ Введите текст		
огурец		
♦ зеруог		

Пример 4.

проц ТЕКСТ_ПО_СЛОВАМ (ТЕКСТ)	\	печать текста по словам
ТЕКСТ:= ТЕКСТ + " "	\	- это - для обработки
	\	последнего слова
пока ТЕКСТ /= "" цикл		
К:= индекс(" ", ТЕКСТ)	\	номер первого вхождения
	\	" " в ТЕКСТ (или 0)
если К /= 1 то		
вывод: ТЕКСТ[:К-1]	\	"аб вг д"[:2] = "аб"
все		

```

ТЕКСТ:= ТЕКСТ[K+1:]
кц
\ "аб вг д"[4:] = "вг д"
конец
\ "аб вг д"[4:5] = "вг"
ТЕКСТ_ПО_СЛОВАМ

```

("Каждый коротышка был ростом с небольшой огурец")

- ◆ Каждый
- ◆ коротышка
- ◆ был
- ◆ ростом
- ◆ с
- ◆ небольшой
- ◆ огурец

```

Кортежи: 4 2, 3, 5, 7, 11 >
          4 "а", "е", "и", "о" >
          4 4 1, 4, 4 6, 7, 9 > >, 10, 4 44, 70, 99 > >

```

Кортеж - упорядоченная последовательность элементов. Элементом кортежа может быть любой объект языка (в том числе и кортеж). Кортеж строится из объектов с помощью операции формирования $\langle \rangle$. Операции над кортежами: # (длина), + (склеивание), * (умножение на целое неотрицательное число), [] (выборка элемента кортежа), [:] (вырезка подкортежа).

Пример 5.

```

функ ТЕКСТ_НА_СЛОВА (ТЕКСТ)
\ разбивка текста на слова
ТЕКСТ:= ТЕКСТ + " "
СЛОВА:= <>
\ пустой кортеж
пока ТЕКСТ /= "" цикл
  К:= индекс(" ", ТЕКСТ)
  если К /= 1 то
    СЛОВА:= СЛОВА + < ТЕКСТ[:К-1] >
    \ < "аб", "вг" > + < "д" > =
    \ < "аб", "вг", "д" >
  все
  ТЕКСТ:= ТЕКСТ[K+1:]
кц
возврат СЛОВА
конец
КОРТЕЖ_ИЗ_СЛОВ:= ТЕКСТ_НА_СЛОВА
("А сам город назывался Цветочным городом")
вывод: КОРТЕЖ_ИЗ_СЛОВ
◆ < "А", "сам", "город", "назывался", "Цветочный", "городом" >

```

Пример 6.
проц СОРТИРОВКА (<=ЧИСЛА)

\ сортировка кортежа чисел
\ <=ЧИСЛА - возвратный (т.е.
\ входной-выходной) параметр

НОВ:= « »

пока ЧИСЛА /= « » цикл

MIN:= ЧИСЛА[1]; ИНД:= 1

для К от 2 до #ЧИСЛА цикл

если ЧИСЛА[К]<MIN то

MIN:= ЧИСЛА[К]; ИНД:= К

все

кц

НОВ:= НОВ + « MIN »

ЧИСЛА[ИНД:ИНД]:= « »

кц

ЧИСЛА:= НОВ

\ « 5, 31, 8 »[1] = 5

\ #« 5, 31, 8 » = 3

\ « » + « 5 » = « 5 »

\ можно иначе:

\ ЧИСЛА := ЧИСЛА[ИНД-1] + ЧИСЛА[ИНД+1:]

\ « 5, 31, 8 »[2:] = « 5, 31 »

\ « 5, 31, 8 »[2:1] = « 31, 8 »

\ « 5, 31, 8 »[1:1] = « 5 »

конец

ДНИ:= « »

повтор 10 цикл

ДНИ:= ДНИ +

« цсч(31) » \ цсч - датчик целых случайных чисел

кц

вывод: ДНИ

« 22, 1, 26, 3, 26, 17, 31, 6, 29, 24 »

СОРТИРОВКА(<=ДНИ) \ <=ДНИ - фактический возвратный параметр

вывод: ДНИ

« 1, 3, 6, 17, 22, 24, 26, 26, 29, 31 »

Пример 7.

функ РЕШЕТО (N)

ПРОСТОЕ:= « да » * N

\ простые числа от 2 до N

\ « да » * 5 =

\ « да, да, да, да, да »

для К от 2 до корень(N)+0.5 цикл

если ПРОСТОЕ[К] то

для J от K+K до N шаг К цикл

ПРОСТОЕ[J]:= нет

\ « да, да, да, нет, да »

кц

все

кц

```

ПРОСТЫЕ_ЧИСЛА := << >
для K от 2 до N цикл
    если ПРОСТОЕ[K] то
        \ << да, да, да, нет, да > [4]
        \
        ПРОСТЫЕ_ЧИСЛА :=
            ПРОСТЫЕ_ЧИСЛА + << K >
        \ << 2, 3 > + << 5 > =
        \ << 2, 3, 5 >
    все
кц
возврат ПРОСТЫЕ_ЧИСЛА
конец
вывод: РЕШЕТО(5)
♦ << 2, 3, 5 >
вывод: РЕШЕТО(200)
♦ << 2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 5
♦ 3, 59, 61, 67, 71, 73, 79, 83, 89, 97, 101, 103, 107, 109,
♦ 113, 127, 131, 137, 139, 149, 151, 157, 163, 167, 173, 179,
♦ 181, 191, 193, 197, 199 >

```

1.3. Переменные

Значением переменной может быть любой объект языка.

Пример 8.

Игра: машина пытается с помощью вопросов угадать задуманного человека или зверя. (Человек отвечает на вопросы "да" или "нет"). Если зверь машине неизвестен, то она с помощью человека пополняет свои знания.

А л г о р и т м. Знания о животных представляются в виде дерева:



Человеку задается вопрос. В зависимости от ответа выбирается правое или левое поддерево. Так до тех пор, пока не будет выбрано название зверя:

зверь

Если это не тот зверь, то этот лист заменяется на поддерево:

вопрос

или

вопрос

/

\

/

\

зверь

зверь-

зверь

зверь

-загадка

-загадка

Загаданный зверь и вопрос, по ответу на который он распознается, запрашиваются у человека.

Р е а л и з а ц и я. Дерево представляется кортежем ЗНАНИЯ:

◀ вопрос, поддерево для 'да', поддерево для 'нет' ▶

Каждое поддерево — это такой же кортеж или текст — название зверя. При выборе поддерева значение переменной ПОПЫТКА — кортеж или текст, в зависимости от глубины спуска по дереву.

проц ЗВЕРИ (<=ЗНАНИЯ)

\■ игра: машина угадывает

\■ задуманного зверя

вывод: ЗНАНИЯ[1], "(да,нет)"

\■ машина задает вопрос

ввод: ОТВЕТ

\■ ввод объекта в диалоге

если ОТВЕТ то

Н:= 2

иначе

Н:= 3

все

ПОПЫТКА:= ЗНАНИЯ[Н]

\■ сокращение вариантов

если тип_корт(ПОПЫТКА) то

\■ еще есть выбор

ЗВЕРИ(<=ПОПЫТКА)

иначе

\■ остался один зверь

вывод: "Это ", ПОПЫТКА, "? (да,нет)"

ввод: ОТВЕТ

если ОТВЕТ то

\■ зверь угадан

вывод: "Ура!"

иначе

\■ этот зверь неизвестен

\■ надо пополнить знания

вывод: "Сдаюсь. Что это за зверь?"

ввод текста: ЗВЕРЬ

вывод: "Введите вопрос, с помощью"

вывод: "которого можно отличить"

```

вывод: ЗВЕРЬ, " и ", ПОПЫТКА, ":"
ввод текста: ВОПРОС
вывод: "Ответ для ", ЗВЕРЬ, "?"
ввод: ОТВЕТ
если ОТВЕТ то
    ПОПЫТКА:= « ВОПРОС, ЗВЕРЬ, ПОПЫТКА »
иначе
    ПОПЫТКА:= « ВОПРОС, ПОПЫТКА, ЗВЕРЬ »
все
все
все
ЗНАНИЯ[N]:= ПОПЫТКА
конец
ВСЕ_ЗВЕРИ:= « "Живет в воде?", "рыба", "страус" »
цикл
    ЗВЕРИ(<=ВСЕ_ЗВЕРИ)
кц
    ♦ Живет в воде?(да,нет)
    нет
    ♦ Это страус?(да,нет)
    нет
    ♦ Сдаксь. Что это за зверь?
    черепаха
    ♦ Введите вопрос, с помощью
    ♦ которого можно отличить
    ♦ черепаха и страус:
    Есть крылья?
    ♦ Ответ для черепаха?
    нет
    ♦ Живет в воде?(да,нет)
    нет
    ♦ Есть крылья?(да,нет)
    да
    ♦ Это страус?(да,нет)
    нет
    ♦ Сдаксь. Что это за зверь?
    попугай
    ♦ Введите вопрос, с помощью
    ♦ которого можно отличить
    ♦ попугай и страус:
    Летать умеет?

```

♦ Ответ для полугай?

да

♦ Живет в воде? (да, нет)

да

♦ Это рыба? (да, нет)

да

♦ Ура!

...

В обозначении переменных можно использовать выборки и вырезки:

ЗНАНИЯ[N]:= ПОПЫТКА \ см. Пример 8

Пример 9.

проц ЗАМЕНА (<=ГДЕ, =>ЧТО, =>НА_ЧТО) \ замена в тексте

\ <=ГДЕ - возвратный параметр,

\ =>ЧТО, =>НА_ЧТО - входные параметры

N:= 1

цикл

\ бесконечный цикл

K:= индекс (ЧТО, ГДЕ [N:1])

\ поиск образца там, где еще не заменяли

если K = 0 то

выход

\ выход из цикла

все

K:= N+K-1

ГДЕ [K:K+#ЧТО-1]:= НА_ЧТО

\ замена образца

\ на новый текст

N:= K + #НА_ЧТО

кц

конец

СКАЗКА:= "... самым известным среди них был малыш " +

"по имени Знайка... Этот Знайка носил яркую голубую " +

"шляпу, желтые, канареечные, брюки и оранжевую " +

"рубашку с зеленым галстуком..."

ЗАМЕНА (<=СКАЗКА, =>"Знайка", =>"Незнайка")

\ <=СКАЗКА - фактический возвратный параметр

\ =>"Знайка", =>"Незнайка" -

\ фактические входные параметры

вывод: СКАЗКА

♦ ... самым известным среди них был малыш по имени Незнайка..

♦ . Этот Незнайка носил яркую голубую шляпу, желтые, канарееч

♦ ные, брюки и оранжевую рубашку с зеленым галстуком...

1.4. Предписания

Предписания: присваивание, вызов процедуры, условное предписание, выборы, циклы, вывод, ввод, выход из цикла, возврат из процедуры или из функции. Почти все предписания уже встречались в приведенных примерах. Еще один пример для завершения картины.

Пример 10. игра в угадывание чисел

```

проц ИГРА ()
  чужие: НАЧАЛО_УГАДАЙКИ, ВАШ_ХОД, МОЙ_ХОД, НЕВЕРНО
  свои: ЗАГАДКА, НАЧ, КОН, ВЫ_УГАДАЛИ, Я_УГАДАЛА
  вывод: "Кто раньше угадает?"
  НАЧАЛО_УГАДАЙКИ()
  цикл
    вывод: "Ваш ход:"; ВАШ_ХОД()
    вывод: "Мой ход:"; МОЙ_ХОД()
    выбор \ выбор истинного условия
      при ВЫ_УГАДАЛИ и Я_УГАДАЛА:
        вывод: "Ничья!"
      при ВЫ_УГАДАЛИ:
        вывод: "Вы выиграли!"
      при Я_УГАДАЛА:
        вывод: "Я выиграла!"
    все
      если ВЫ_УГАДАЛИ или Я_УГАДАЛА то
        возврат \ возврат из процедуры
      все
    кц по НЕВЕРНО() \ конец цикла, если
    \ НЕВЕРНО() = да
  вывод: "Вы где-то напутали!"
  вывод: "Я так вольше не играю."
конец
проц НАЧАЛО_УГАДАЙКИ ()
  чужие: ЗАГАДКА, НАЧ, КОН, ВЫ_УГАДАЛИ, Я_УГАДАЛА
  вывод: "Задумайте число от 1 до 1000"
  вывод: "и угадайте мое"
  ЗАГАДКА := псч(1000)
  НАЧ := 0
  КОН := 1001
  ВЫ_УГАДАЛИ := нет
  Я_УГАДАЛА := нет

```



```

конец
проц ВАШ_ХОД ()
  чужие: ЗАГАДКА, ВЫ_УГАДАЛИ
  свои: ОТГАДКА
  ввод: ОТГАДКА
  выбор
    при ОТГАДКА>ЗАГАДКА:
      вывод: "Мое число меньше!"
    при ОТГАДКА<ЗАГАДКА:
      вывод: "Мое число больше!"
  все
  ВЫ_УГАДАЛИ:= (ОТГАДКА = ЗАГАДКА)
конец
проц МОЙ_ХОД ()
  чужие: НАЧ, КОН, Я_УГАДАЛА
  свои: ОТГАДКА, ОТВЕТ, ОТВЕТ_ВЕРЕН
  ОТГАДКА:= (НАЧ+КОН) // 2
  вывод: "Это ", ОТГАДКА, "?"
  цикл
    вывод: "Ответы: =, > (загадка)", ОТГАДКА,
      ")", < (загадка<", ОТГАДКА, ")"
    ввод текста: ОТВЕТ
    ОТВЕТ_ВЕРЕН:= да
    выбор ОТВЕТ
      при ">": НАЧ := ОТГАДКА
      при "<": КОН := ОТГАДКА
      при "=": Я_УГАДАЛА:= да
      иначе ОТВЕТ_ВЕРЕН:= нет; вывод: "Вы ошиблись"
    все
  кц по ОТВЕТ_ВЕРЕН
конец
функ НЕВЕРНО ()
  чужие: КОН, НАЧ
  возврат НАЧ+1=КОН
конец
ИГРА()
  ♦ Кто раньше угадает?
  ♦ Задумайте число от 1 до 1000
  ♦ и угадайте мое
  ♦ Ваш ход:
512
  ♦ Мое число меньше!

```

♦ Мой ход.
 ♦ Это 500?
 ♦ Ответы: =, > (загадка > 500), < (загадка < 500)
 <
 ♦ Ваш ход:
 256
 ♦ Мое число больше!
 ♦ Мой ход.
 ♦ Это 250?
 ♦ Ответы: =, > (загадка > 250), < (загадка < 250)
 <
 ♦ Ваш ход:
 384
 ♦ Мой ход.
 ♦ Это 125?
 ♦ Ответы: =, > (загадка > 125), < (загадка < 125)
 >
 ♦ Вы выиграли!

1.5. Процедуры и функции

В процедуре (функции) все нелокальные имена должны быть перечислены в списке "чужие". (Локальные имена можно перечислить в списке "свои", но это не обязательно). Имена, используемые в данной процедуре, известны и в вызываемых из нее процедурах (если там они указаны как "чужие"), так как поиск "чужих" имен ведется по динамической цепочке вызовов (см. Пример 10).

Параметры процедур:

входные:	=>ЧТО	\ см. Пример 9
	ТЕКСТ	\ см. Пример 4
	(стрелка => не обязательна)	
возвратные:	<=ГДЕ	\ см. Пример 9
	<=ЗНАНИЯ	\ см. Пример 8

У функций параметры только входные.

1.6. Модули и устройства

Планируемые возможности аппарата модулей и устройств:

– подключение модуля пополняет список стандартных имен

(в том числе имен стандартных процедур и функций);

- подключение модуля пополняет набор простых предписаний (т.е. языковыми средствами создаются исполнители, аналогичные существующим в Ровике [1]);
- устройство есть частный случай модуля. С устройством возможен обмен через предписания ввода/вывода:

вывод >>принтер: "Этот текст - на принтер"

В предписаниях ввода/вывода могут быть указаны различные режимы ввода/вывода:

ввод текста: ВОПРОС

\ см. Пример 8

"текста" - ключевое слово для устройства КЛАВ (клавиатура).

2. Рапира. Формальное описание языка (без модулей, устройств и обработки исключительных ситуаций)

2.1. Синтаксис

Алфавит:

- буквы: большие и малые, русские и латинские;
- цифры;
- спецзнаки:

+ , : ; () [] < > =
+ - * / % # _ " \ « »

- и все, что угодно.

разделитель ::= (пробел ::= ' ') ;
(перевод_строки ::= '↵') ;
(комментарий ::= '\{ литера* } '↵')

лексема ::= спецсимвол ;
ключевое_слово ;
имя ;
целое_без_знака ;
вещественное_без_знака ;
текст

спецсимвол ::= : , : ; () [] « »
:= => < > <= >= = /= #
+ - * / // /% %%

* кроме '↵'

```

ключевое_слово ::= влс      вызов      или      по      текста
                  вввод      выход      иначе    повтор   то
                  возврат    для      конец    пока      функ
                  все        до        кц       при      цикл
                  выбор      если     не       проц     чужие
                  вывод      и         от       свои     шаг

имя ::= буква { буква ; '_' ; цифра }
целое ::= [ '+' ; '-' ] целое_без_знака
целое_без_знака ::= цифра { цифра }
вещественное ::= [ '+' ; '-' ] вещественное_без_знака
вещественное_без_знака ::= цифра { цифра }
                        ( '.' цифра { цифра } [ 'e' целое ] ;
                        'e' целое )

текст ::= '""' { литера* ; '""' } '""'
условие ::= выражение ::=
    вып_а ( ( 'и' ; 'или' ) вып_а )
вып_а ::=
    [ 'не' ]
    вып_б
    { ( '>' ; '<' ; '>=' ; '<=' ; '=' ; '/=' )
      вып_б }

вып_б ::=
    [ '+' ; '-' ]
    вып_в
    { ( '^' ; '*' ; '/' ; '//' ; '%' ; '+' ; '-' )
      вып_в }

вып_в ::=
    [ '#' ]
    ( имя ;
      текст ;
      целое_без_знака ;
      вещественное_без_знака ;
      процедура ;
      функция ;
      ( фори_корт ::= '←' [ л_выр ] '→' ) ;
      ' ( выражение ) '
      { ( выв_выр ::= '[' л_выр ']' ;
        '[' выражение ':' ;
        выражение ']' ) }
      факт_парам_функ
    )

```

* кроме '""' и '←'


```

л_выр ::= выражение ( ',' выражение )
факт_парам_функ ::=
    '(' [ [ '=' ] выражение
        { ',' [ '=' ] выражение } ] ')'
процедура ::=
    'проц' [ имя ]
    '(' [ [ '=' | '<=' ] имя
        { ',' [ '=' | '<=' ] имя } ] ')' ;
    [ описание_инен ]
    предп
    'конец'
;; ::=
    ( ';' | '←' )
описание_инен ::= ( 'чужие' ':' имя ( ',' имя ) ;
    [ 'свои' ':' имя ( ',' имя ) ; ] ) ;
    ( 'свои' ':' имя ( ',' имя ) ;
    [ 'чужие' ':' имя ( ',' имя ) ; ] )
функция ::=
    'функ' [ имя ]
    '(' [ [ '=' ] имя
        { ',' [ '=' ] имя } ] ')' ;
    [ описание_инен ]
    предп
    'конец'
единица_общения_с_системой ::=
    ( предписание ; описание ) ;
предп ::=
    ( предписание ; )
описание ::=
    ( процедура ; функция )
предписание ::=
    ( ( пустое ::= '' ) ;
    присваивание ;
    вызов_процедуры ;
    условное ;
    выбор ;
    цикл ;
    вывод ;
    ввод ;
    ( выход_из_цикла ::= 'выход' ) ;
    ( выход_из_процедуры ::= 'возврат' ) ;
    ( выход_из_функции ::= 'возврат' выражение ) )
присваивание ::=
    ( переменная ::= имя ( выр_выр ) ) ':'= выражение

```

```

вызов_процедуры ::=
    ( 'вызов' выражение ; имя ) факт_парам_проц
факт_парам_проц ::=
    ' (' [
        ( ' <=' переменная ;
          [ '=' ] выражение )
        { ',' ( ' <=' переменная ;
          [ '=' ] выражение ) } ] ')'
условное ::=
    'если' условие
    'то' предп
    [ 'иначе' предп ]
    'все'
выбор ::=
    (
        'выбор' выражение
        ( 'при' выражение { ',' выражение } ':' предп )
    )
    'выбор'
    ( 'при' условие ':' предп )
    [ 'иначе' предп ]
    'все'
цикл ::=
    [ ( 'для' имя [ 'от' выражение ]
        [ 'до' выражение ]
        [ 'шаг' выражение ] ;
        'повтор' выражение ) ]
    [ 'пока' условие ]
    'цикл'
    предп
    'кц' [ 'по' условие ]
вывод ::=
    'вывод' [ 'вс' ] [ ':' выражение { ',' выражение } ]
ввод ::=
    'ввод' [ 'текста' ] ':' переменная { ',' переменная }

```

Контекстные условия :

- предписание "выход_из_цикла" может стоять только в цикле;
- предписание "выход_из_процедуры" может стоять только в процедуре;
- предписание "выход_из_функции" может стоять только в

функции;

- в процедуре и функции : все имена в списках "чужие", "свои", в списке параметров и имя процедуры (функции) - все различны;
- в "описании" (см. "единица_общения_с_системой") имя процедуры (функции) обязательно.

2.2. Семантика

2.2.1. Имена и переменные. Значением переменной может быть любой объект языка. Существуют стандартные имена (см. п.2.3). Правила локализации имен см. п.2.2.2 (процедуры и функции). Если имя в данной области локализации не является стандартным и ему не было присвоено значение, то его значением является пустой объект (пусто).

2.2.2. Объекты.

объект:	обозначение_объекта::=	
пустой	"пусто"	:
логический	"да" ; "нет"	:
процедура	процедура	/X см. синтаксис X/ :
функция	функция	/X см. синтаксис X/ :
целый	целое ;	
	целое_без_знака	/X см. синтаксис X/ :
вещественный	вещественное ;	
	вещественное_без_знака	/X см. синтаксис X/ :
текст	текст	/X см. синтаксис X/ :
кортеж	" « [обозначение_объекта { ', ' обозначение_объекта ?] » "	

Текст. Последовательность " " " в обозначении текста представляет литеру " " " в тексте.

Кортеж - упорядоченный набор произвольных объектов.

Процедуры и функции. Имя процедуры (функции) локально в процедуре (функции).

Параметры: входные - ['='] имя
и возвратные - '<=' имя .

У функций параметры только входные. Описания имен: по умолчанию имя, не являющееся стандартным, считается "своим" (локальным). Правило локализации имен: по динамической цепочке вызовов.

2.2.3. Приоритеты операций (по убыванию):

◀ ▶ () [] [:] () - вызов функции

#

%%

* / // %

- + (унарные и бинарные)

> < >= <=

= /=

ne

и

или

2.2.4. Операции, предписания и описания. Семантика большинства конструкций языка описана в терминах самого языка с использованием базисного набора предписаний и операций. Семантика базисного набора описана словесно.

2.2.4.1. Обозначения.

a, b, c, z, b_j, f, f_j, g, k, h, p, n, x - выражения

i

- имя

v

- переменная

S, S_j

- предписания

2.2.4.2. Базисный набор.

Операции:

p(...) - p функция

- вызов функции.

◀ ... ▶

- формирование кортежа.

#k

- k текст, кортеж

- длина.

k [a]

- k текст, кортеж;

a целое.

1 ≤ a ≤ #k. - выборка.

результат:

элемент кортежа с номером a

(символ текста с номером a)

a %% b

- a, b - числа

- возведение в степень.

b - вещественное: a %% b = exp(ln(a) * b)

b - целое:

см. частичное сведение

k базису.

a * b

- a, b - числа;

a текст, кортеж, b целое:

b текст, кортеж, a целое

- умножение.

умножение на целое число

		см. частичное сведение к базису.
a / b	- a, b - числа	- деление.
		результат:
	a, b - целые:	целый или вещественный,
	в остальных случаях - вещественный.	
$-a$	- a число	- противоположное значение.
$a + b$	- a, b числа;	
	a, b тексты;	
	a, b кортежи	- сложение чисел, склеивание текстов и кортежей.
$a > b$	- a, b числа	- сравнение "больше".
$a = b$	- a, b - любые	- сравнение на равенство;
		случай a, b - кортежи см. частичное сведение к базису.

Предписания:

$i := x$	- x левый объект	- присваивание значения к имени i .
вызов $p(\dots)$	- p процедура	- вызов процедуры.
если f то S else	- f логическое	- условное предписание.
вывод: ...		- вывод объектов. выводятся (на экран) обозначения_объектов (кроме текстов, которые выводятся без кавычек).
ВЫВОД ВПС: ...		- вывод без перевода строки (здесь временно, потом в устройстве)
ввод: ...		- ввод объектов. вводятся (с клавиатуры) обозначения_объектов.
ввод текста: ...		- ввод текста без кавычек (здесь временно, потом в устройстве).
возврат		- возврат из процедуры.
возврат x	- x левый объект	- возврат значения из функции.

2.2.4.3. Конструкции, сводимые к базису. Определяющие конструкции построены так, что для неверных аргументов дают ошибку, причем не позже, чем необходимо. Все новые имена дол-

жны отличаться от имен в смысле Рапиры (в приводимых фрагментах это не так для удобства чтения).

Дополнительно использованы:

- процедура ОШИБКА (она могла бы быть и с параметром - какая ошибка),
- стандартные функции ТИП_ЦЕЛ, ...
- логические предписания "ЗАЩИТИТЬ имя" (т.е. запретить присваивания этому имени) и "СНЯТЬ ЗАЩИТУ имя", без которых не описывается цикл ДЛЯ.

Частичное сведение к базису.

1. Равенство для кортежей:

```
k = h      ≡
      функ РАВ_КОРТ (K, H)      \ рекурсивное место
      если #K /= #H то
      возврат нет
      все
      для ИНД до #K цикл
      если K[ИНД] /= H[ИНД] то
      возврат нет
      все
      кц
      возврат да
      конец
      (k, h)
```

2. Умножение на целое число

(текста или кортежа - только на неотрицательное число):

```
a * h      ≡
      функ ЦЕЛ_УМН (AA, BB)
      если тип_цел(AA) то
      A:= AA ; B:= BB
      иначе
      A:= BB ; B:= AA
      все
      выбор
      при тип_цел(B) :   ПР:= 0
      при тип_вещ(B) :   ПР:= 0.0
      при тип_текст(B) : ПР:= ""
      при тип_корт(B) :   ПР:= {}
      все
      если A>=0 то  Y:=  A
      иначе        Y:= -A
      все
```

```

повтор У цикл
    ПР := ПР + В
кц
если A >= 0 то возврат ПР
иначе      возврат -ПР \ для текста или
все        \ кортежа - ошибка
конец
(a, b)

```

3. Возведение в целую степень:

$a \times b$

```

≡
функ ЦЕЛ_СТЕП (A, B)
    если A = 0 и B = 0 то
        ошибка()
    все
    если B > 0 то N := B
    иначе      N := -B
    все
    P := 1
    пока N > 0 цикл
        если N / 2 /= 0 то
            P := P * A
        все
        A := A * A
        N := N / 2
    кц
    если B > 0 то возврат P
    иначе      возврат 1/P
    все
конец
(a, b)

```

Сведение конструкций к базисным.

Предписания:

1) вызов процедуры:

$i(\dots) \equiv \text{вызов } i(\dots)$ \ i — процедура

2) условное:

если f то $S1$

иначе $S2$

все

вызов

проц ЕСЛИ_ИНАЧЕ (F)

чужие : ... \ все имена из $S1$, $S2$ и f

если F то

```

        S1
        возврат
    все
    S2
конец
(f) \ f - логическое

3) выбор:
выбор
    при f1 : S1
    при f2 : S2
    ...
    при fn : Sn
    иначе S0
все
=
если f1 to S1
иначе если f2 to S2
иначе ...
    если fn to Sn
иначе S0
все все ... все

выбор a
    при b1 : S1
    при b2 : S2
    ...
    при bn : Sn
    иначе S0
все
=
вызов
проц ВЫБОР_ВЫР (A)
    чужие : ... \ все имена из S1, b1 и a
    если A = b1 to S1
    иначе если A = b2 to S2
    иначе ...
        если A = bn to Sn
    иначе S0
    все все ... все
конец
(a) \ a - логическое

выбор a
    при b1, b2 : S1
    ...
все
=

```



```

выбор a
при b1: S1
при b2: S1
...
все
Операции:
4) не f
≡
функ ЛОГ_НЕ (F)
если F то возврат да
иначе возврат нет
все
конец
(f)

5) f и g
≡
функ ЛОГ_И ( )
чужие: ... \ все имена из f, g
если не f то
возврат нет
иначе
если g то возврат да
иначе возврат нет
все
конец
( ) \ f, g логические

6) f или g
≡
функ ЛОГ_ИЛИ ( )
чужие: ... \ все имена из f, g
если f то
возврат да
иначе
если g то возврат да
иначе возврат нет
все
конец
( ) \ f, g логические

7) +a
≡ 0+a
8) a - b
≡ a + (-b)

```

9) $a < b$

≡

ФУНК МЕНЬШЕ (A, B)

возврат не (A>B или A=B)

конец

(a, b)

10) $a \geq b$

≡

не (a<b)

11) $a \leq b$

≡

не (a>b)

12) $a \neq b$

≡

не (a=b)

Предписания:

13) цикл, выход из цикла:

цикл

S

кц

≡

вызов

проц БЕСКОНЕЧНО ()

чужие: ...

\ все имена из S

S

БЕСКОНЕЧНО ()

конец

не ()

выход

≡

возврат

\ если цикл — как выше

пока f цикл

S

кц по g

≡

цикл

если не (f) то выход все

S

если g то выход все

кц

пока f цикл

S

кц

≡

пока f цикл

S

кц по нет

цикл

S

кц по f

≡

пока да цикл

S

кц по f

```

повтор n
  пока f цикл
    S
кц по g ≡
  вызов
    проц ЦИКЛ_ПОВТОР (N)
    чужие: ... \ все имена из S, f и g
    если не(тип_цел(N) и N>=0) то
      ошибка()
    все
    пока N>0 и f цикл
      S
      N:= N-1
    кц по g
  конец
  (n) \ n целое

повтор n цикл
  S
кц ≡
  повтор n
    пока да цикл
      S
    кц по нет
  для i от a шаг c
    пока f цикл
      S
    кц по g ≡
    вызов
      проц ЦИКЛ_ДЛЯ_БЕЗ_ДО (A, C)
      чужие: ... \ i, все имена из S, f и g
      если не(тип_цел(A) или тип_вещ(A)) то
        ошибка()
      все
      если не(тип_цел(C) или тип_вещ(C)) то
        ошибка()
      все
      если C=0 то ошибка() все
      i:= A
      защитить i
      пока f цикл
        S
      снять защиту i

```

```

        i := i + C
        защитить i
        кц по g
        снять защиту i
        конец
        (a, c) \ a, c - числа

для i от a шаг c цикл
    S
кц
    ≡
    для i от a шаг c
        пока да цикл
            S
        кц по нет
    для i шаг c цикл
        S
    кц
    ≡
    для i от 1 шаг c цикл
        S
    кц
    для i от a цикл
        S
    кц
    ≡
    для i от a шаг 1 цикл
        S
    кц
    для i от a до b шаг c
        пока f цикл
            S
    кц по g
    ≡
    вызов
    проц цикл_для_до (A, B, C)
    чужие: ... \ i, все имена из S, f, g
    если не(тип_цел(B) или тип_вещ(B)) то
        ошибка()
    все
    для i от A шаг C
        пока ((B-i) * C >= 0) и f цикл
            S
        кц по g
    конец
    (a, b, c) \ a, b, c - числа

```


для i от a до b шаг c цикл

S

кц

для i от a до b шаг c

пока да

цикл

S

кц по нет

Операции:

14) $k[a, b, \dots, z] \equiv k[a][b] \dots [z]$

15) $k[a : b]$

$k[a : b] \equiv$

функ ВЫРЕЗКА (K, A, B)

если не тип_цел(B) то

ошибка()

все

если $(B-A+1 < 0)$ то

ошибка()

все

если тип_текст(K) то

N := ""

для ИНД от A до B цикл

N := N + K[ИНД]

кц

иначе

N := « »

для ИНД от A до B цикл

N := N + « K[ИНД] »

кц

все

возврат N

конец

(k, a, b)

\ k - текст или кортеж,

\ a, b - целые

$k[: b] \equiv$

$k[1:b]$

$k[:] \equiv$

k

$k[a :] \equiv$

функ ХВОСТ (K, A)

возврат K[A: #K]

конец

(k, a)

\ k - текст или кортеж,

\ a - целое

16) $a // b$

```
≡
функ ДЕЛЕНИЕ_НАЦЕЛО (A, B)
  ЕСЛИ НЕ (тип_цел(A) И тип_цел(B)) ТО
    ОШИБКА()
  ВСЕ
  ЕСЛИ B <= 0 ТО
    ОШИБКА()
  ВСЕ
  РЕЗ := 0
  ЕСЛИ A >= 0 ТО
    ПОКА A >= B ЦИКЛ
      A := A - B
      РЕЗ := РЕЗ + 1
    КЦ
  ИНАЧЕ
    ПОКА (-A) >= B ЦИКЛ
      A := A + B
      РЕЗ := РЕЗ - 1
    КЦ
  ЕСЛИ A /= 0 ТО РЕЗ := РЕЗ - 1 ВСЕ
  ВОЗВРАТ РЕЗ
конец
```

(a, b) \ a, b целые

17) $a \% b$

```
≡
функ ОСТАТОК (A, B)
  ВОЗВРАТ A - A // B * B
конец
```

(a, b) \ a, b целые

Предписания:

18) $v[a : b] := h$

вызов

```
проц ПРИСВ_ВЫР (<=K, A, B, H)
  ЕСЛИ (B - A + 1 < 0) ТО
    ОШИБКА()
  ВСЕ
  K := K[A-1] + H + K[B+1:]
конец
```

(<= v, a, b, h) \ (v, h - кортежи;
 \ v, h - тексты),
 \ a, b - целые

```

19) v [ a ] := x      ≡
    вызов
    проц ПРИСВ_ВЫБ (<= K, A, X)
    если тип_текст(K) то
        если #X /= 1 то
            ошибка()
        все
            K[A:A] := X
        иначе
            K[A:A] := « X »
        все
    конец
    ( <= v, a, x)
    \ (v - кортеж, x - любой !
    \ v - текст, x - текст длины 1),
    \ a - целое

```

Описания:

20) описание процедуры (см. "единица_общения_с_системой"):

```

    проц i
    ...
    конец      ≡
    i := проц i
    ...
    конец

```

21) описание функции (см. "единица_общения_с_системой"):

```

    функ i
    ...
    конец      ≡
    i := функ i
    ...
    конец

```

2.3. Стандартные имена

Константы:

```

пусто
да
нет
пс (пс = "4")
пи , рi (пи = рi = 3.14...)

```

Функции:

abs (x)	- модуль числа x.
sign(x)	- знак числа x (-1, 0, +1).
sqrt(x), корень(x)	- корень числа x.
целч(x)	- целая часть числа x (целч (5.9) = 5 ; целч (-5.9) = -6 ; целч (5.4) = 5 ; целч (-5.4) = -6).
окрч(x)	- округление числа (окрч (5.9) = 6 ; окрч (-5.9) = -6 ; окрч (5.4) = 5 ; окрч (-5.4) = -5).
дсч (x)	- датчик случайных чисел в интервале [0 - x [(x >= 0).
цсч (x)	- датчик целых случайных чисел в интервале [1 - x] (x >= 0).
индекс(З , K)	- индекс элемента З в кортеже K (подтекста З в тексте K); выдается индекс первого вхождения или 0, если вхождений нет.
тип_пуст(x), тип_лог(x), тип_цел(x), тип_вещ(x), тип_текст(x), тип_корт(x), тип_проц(x), тип_функ(x)	- значение функции - 'да' или 'нет', в зависимости от типа объекта x.
sin (x)	
cos (x)	
tg (x)	
arcsin(x)	
arctg (x)	
exp (x)	
ln (x)	
lg (x)	

Графические процедуры

(значения аргументов xi, yi, r - числа):

тчк (x,y)	- точка.
лин (x1,y1,x2,y2)	- отрезок.
окр (x,y,r)	- окружность.

3.2. Семантика

Объекты. В тексте:

- '^' буква обозначает контрольный символ;
- '\' число обозначает литеру с данным кодом.

Предписания. Ввод – не только обозначений_объектов, а любых выражений, через запятые. "Ввод текста" – ввод только одного текста; "ввод пригл" – ввод с приглашением.

3.3. Стандартные имена

Нет функций "tg", "arcsin".

Дополнительные функции:

- код(t) – код символа t.
- алф(x) – символ с кодом x.
- клав() – ввод одного символа (нажатием одной клавиши);
результат – введенный символ.
- нажато() – проверка, была ли нажата клавиша;
результат – введенный символ или пустой текст.

Графика:

- нет процедуры "обл";
- граничные координаты графического окна:
x-координата : 0 – 255,
y-координата : 0 – 191;
- цвета задаются только числами:
0–16 (0 – пустой, 16 – реверсный).

Л и т е р а т у р а

1. Звенигородский Г.А. Первые уроки программирования. – М.: Наука, 1985.
2. Звенигородский Г.А. и др. Программная система "Школьница" и ее реализация на персональной ЭВМ //Микропроцессорные средства и системы. – 1984. – N 1. – С.50–55.

Л.С.Бараз, Е.В.Боровиков, Н.Г.Глаголева,
П.А.Земцов, Е.В.Налимов, В.А.Шикоза

ЯЗЫК ПРОГРАММИРОВАНИЯ РАПИРА

П р е п р и н т

767

Рукопись поступила в редакцию 13.11.87

Рецензент Т.М. Яхно

Редактор Е.В.Блинова

Подписано в печать 24.12.87г. МН 08961

Формат бумаги 60x84 1/16 Объем 2 уч.-изд.л., 2,1 п.л.

Тираж 350 экз.

Заказ № 4

Ротапринт ВЦ СО АН СССР, Новосибирск 90